1 INTRODUÇÃO E DEFINIÇÃO DO PROBLEMA

No contexto corporativo de Tecnologia da Informação e gestão de redes, a execução de rotinas administrativas via interface de linha de comando (CLI), especificamente em ambientes Microsoft Windows com *PowerShell*, representa um vetor crítico de produtividade e risco operacional. A complexidade sintática de cmdlets avançados, combinada à dispersão documental em repositórios desestruturados, frequentemente induz analistas de suporte a consultar fontes despadronizadas ou genéricas na Internet, conforme apontam as diretrizes de governança em TI (**PRESSMAN; MAXIM, 2016**).

Tal fragmentação acarreta elevação direta no Tempo Médio de Resolução (MTTR) de incidentes, além de expor a infraestrutura a falhas decorrentes da execução inadvertida de comandos com privilégios elevados. O projeto **ShellBase** (**SHELLBASE, 2026**) foi idealizado para sanar essa lacuna, provendo uma plataforma centralizada de busca semântica, síntese de comandos homologados e execução supervisionada amparada por Inteligência Artificial e Arquitetura RAG (*Retrieval-Augmented Generation*).

LINGUAGEM CENTRAL	ALVO OPERACIONAL	INDEXADOR SEMÂNTICO
Python 3.11+	**PowerShell 7 / Win32**	**Vector DB / Embeddings**

2 FUNDAMENTAÇÃO TECNOLÓGICA E LINGUAGEM

A implementação do ShellBase baseia-se primordialmente na linguagem **Python** (**PYTHON SOFTWARE FOUNDATION, 2026**), selecionada em virtude de seu robusto ecossistema de processamento de linguagem natural, suporte assíncrono e fácil orquestração com serviços de banco vetorial. Complementarmente, o **PowerShell** (**MICROSOFT CORPORATION, 2026**) atua como subsistema executor alvo, propiciando integração nativa com o Windows Management Instrumentation (WMI), Active Directory e serviços internos do sistema operacional.

O sistema opera sob o paradigma modular: separação estrita de responsabilidades entre ingestão documental, núcleo de inferência/orquestração lógica e interface de comando unificada.

3 DECOMPOSIÇÃO MODULAR E ESTRUTURAÇÃO DO CÓDIGO

3.1 MÓDULO DE INGESTÃO DOCUMENTAL: INGEST.PY

O script `ingest.py` é responsável pelo pipeline de pré-processamento e vetorização. Ele extrai procedimentos operacionais padrão (POPs), scripts homologados e manuais técnicos, fracionando-os em blocos de contexto semântico (*chunks*) para geração de vetores dimensionais.

Listagem 1 — Rotina de extração e vetorização (ingest.py)

```
      def ingest_documents(docs_path: str, chunk_size: int = 500): """Carrega manuais e gera
embeddings no banco vetorial.""" documents = load_directory_documents(docs_path) chunks =
split_into_chunks(documents, max_chars=chunk_size) for chunk in chunks: vector =
generate_embeddings(chunk.content, model="nomic-embed-text") vector_db.upsert( id=chunk.doc_id,
vector=vector, metadata={"source": chunk.filepath, "content": chunk.content} ) print(f"[OK]
Ingestão concluída: {len(chunks)} fragmentos indexados.")
```

3.2 MOTOR COGNITIVO E RAG: CORE.PY

O arquivo `core.py` encapsula a inteligência da aplicação. Ele recebe a solicitação do usuário, consulta a base vetorial para recuperar os fragmentos mais similares via distância de cosseno e monta o prompt condicionado para o modelo de linguagem (LLM), mitigando riscos de alucinação sintática.

Listagem 2 — Recuperação contextual e geração de comando (core.py)

```
      def retrieve_and_generate(user_prompt: str, top_k: int = 3) -> dict: """Recupera contexto
homologado e gera instrução PowerShell precisa.""" query_vector = generate_embeddings(user_prompt)
relevant_chunks = vector_db.similarity_search(query_vector, limit=top_k) context_text = "\n---
\n".join([doc["content"] for doc in relevant_chunks]) system_prompt = ( "Você é o assistente
técnico ShellBase. Com base estrita no contexto " "fornecido a seguir, gere apenas o comando
PowerShell correspondente:\n" f"CONTEXTO:\n{context_text}" ) raw_response =
llm_inference(prompt=user_prompt, system=system_prompt) return {"command":
sanitize_powershell(raw_response), "context": context_text}
```

3.3 ORQUESTRADOR CENTRAL E EXECUÇÃO: MAIN.PY

O arquivo `main.py` constitui o ponto de entrada (*entry point*) e terminal de controle interativo. Ele estabelece o elo entre a interface do operador e as camadas de ingestão e cognição, implementando o mecanismo de segurança *Human-in-the-Loop* antes do disparo ao subprocesso do sistema operacional.

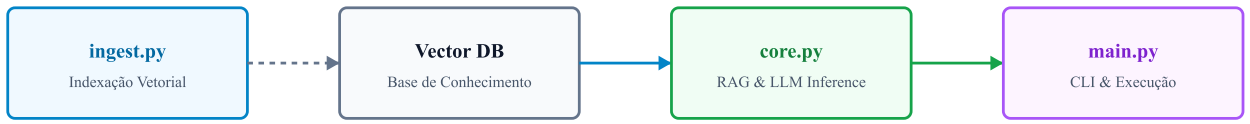
Listagem 3 — Ponto de entrada e ciclo interativo de execução (main.py)

```
      import subprocess from core import retrieve_and_generate from ingest import ingest_documents
def main(): print("=== ShellBase CLI — Assistente de Automação PowerShell ===") while True: prompt
= input("\nShellBase >> ").strip() if prompt.lower() in ("exit", "sair"): break if
prompt.startswith("!ingest"): ingest_documents("./knowledge_base"); continue result =
retrieve_and_generate(prompt) print(f"\n[COMANDO SUGERIDO]:\n {result['command']}\n") confirm =
input("Deseja executar o script no terminal? [s/N]: ") if confirm.lower() == 's': exec_res =
subprocess.run(["powershell", "-Command", result['command']], capture_output=True, text=True)
print(exec_res.stdout if exec_res.returncode == 0 else exec_res.stderr)
```

3.4 INTERAÇÃO E FLUXO DE CONTROLE ENTRE OS MÓDULOS

A comunicação orquestrada em `main.py` viabiliza um ciclo coeso: ao acionar a rotina de atualização (`!ingest`), delega a carga para `ingest.py`. Durante consultas em linguagem natural, `main.py` requisita a resolução semântica a `core.py`, que recupera a base vetorial, formata a chamada e devolve o comando higienizado para validação pelo operador humano.

Figura 1 — Diagrama de fluxo de integração entre os componentes



4 CONSIDERAÇÕES DE SEGURANÇA E CONCLUSÃO

A implementação do ShellBase valida a hipótese de que a arquitetura RAG reduz substancialmente a incidência de comandos errôneos em tarefas administrativas de sistemas operacionais. Ao impedir execuções automatizadas sem supervisão e manter o operador humano no laço decisório (*Human-in-the-Loop*), o sistema combina agilidade de recuperação semântica com os parâmetros de conformidade da norma **ABNT NBR 14724 (2011)** e governança corporativa de TI.

O desacoplamento entre `ingest.py`, `core.py` e `main.py` assegura escalabilidade e manutenibilidade, permitindo que a solução receba novos conectores de dados ou motores de inferência locais sem necessidade de refatoração do subsistema executor.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: Informação e documentação — Trabalhos acadêmicos — Apresentação. Rio de Janeiro: ABNT, 2011.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: Informação e documentação — Referências — Elaboração. Rio de Janeiro: ABNT, 2018.

MICROSOFT CORPORATION. **PowerShell Documentation**: Automating tasks with cross-platform command-line shell. Redmond: Microsoft, 2026. Disponível em: <https://learn.microsoft.com/powershell/>.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de Software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

PYTHON SOFTWARE FOUNDATION. **Python Language Reference**: Version 3.11+. Wilmington: Python Software Foundation, 2026. Disponível em: <https://www.python.org/>.

SHELLBASE. **ShellBase: Assistente RAG e automação para rotinas em PowerShell**. Desenvolvido por IgnIT0. Repositório de código-fonte no GitHub, 2026. Disponível